

EOD – AWS WAF & AWS Systems Manager Parameter Store

Date: 30 September 2026

Environment: AWS

Region: US East (N. Virginia) – us-east-1

1. Objective

The objective of this hands-on activity was to understand and implement:

1. **AWS Web Application Firewall (AWS WAF)**
2. **AWS Systems Manager Parameter Store**

The activity focused on understanding the concepts, practical implementation, configuration, security controls, and basic validation.

2. AWS WAF

2.1 What is AWS WAF?

AWS WAF (Web Application Firewall) is a web application security service used to monitor and control HTTP/HTTPS requests before they reach an application.

It can help protect applications against common web threats such as:

- SQL injection
- Malicious inputs
- Unwanted IP addresses

- Anonymous IP traffic
- Excessive request rates
- Common web attacks
- Certain bot traffic patterns

AWS WAF can be associated with services such as CloudFront, Application Load Balancer, API Gateway, and other supported AWS resources.

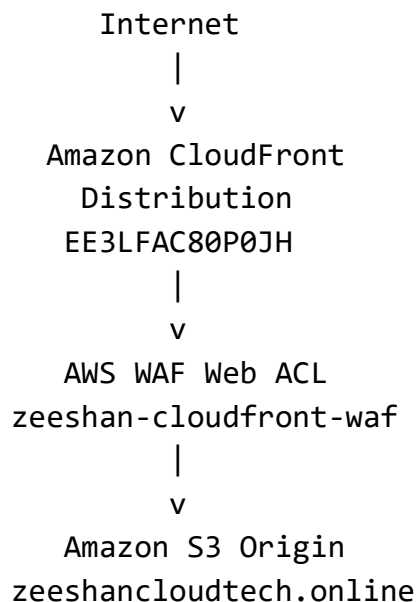
2.2 Practical Scenario

For this lab, an existing static website was being served using:

- Amazon S3
- Amazon CloudFront
- Custom domain: `zeeshancloudtech.online`

The objective was to add AWS WAF protection in front of the CloudFront distribution.

Architecture



3. AWS WAF Implementation

3.1 Web ACL / Protection Pack Created

Created the following WAF protection pack:

Name:

zeeshan-cloudfront-waf

Scope:

CloudFront (Global)

Associated Resource:

EE3LFAC80P0JH - zeeshancloudtech.online

Web ACL ID:

8fb4b209-0838-4644-8b02-54b2a2cadd48

3.2 WAF Rules

The created protection pack contains **19 rules**.

Important rules include:

AWS Managed Rules

- AWS Managed Rules Anti-DDoS Rule Set
- Amazon IP Reputation List
- Anonymous IP List
- Common Rule Set
- Known Bad Inputs Rule Set
- PHP Rule Set
- SQL Injection Rule Set

- Admin Protection Rule Set
- Bot Control Rule Set

Custom / Protection Rules

- IPv4 Allow Rule
- IPv6 Allow Rule
- IPv4 Block Rule
- IPv6 Block Rule
- Geographic Restriction Rule
- Global Rate-Based Rule
- GET Rate-Based Rule
- POST/PUT/DELETE Rate-Based Rule
- Body Size Restriction Rule
- Managed IP DDoS Rate Limit

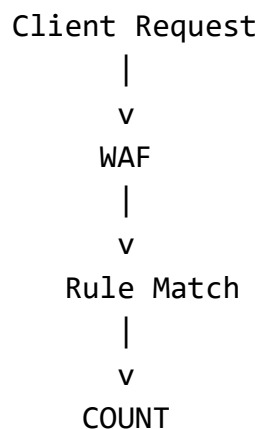
3.3 Count Mode

Several rules are configured in **Count mode**, including rate-based and body-size-related rules.

Count mode allows requests to be monitored without immediately blocking them.

This is useful when introducing a WAF rule because traffic behavior can be observed before changing the rule to an enforcing action.

Example:



Request continues

The screenshot shows the AWS CloudFront console with the 'Distributions' page selected. The left sidebar contains navigation links for CloudFront, Policies, Functions, Static IPs, VPC origins, SaaS, and Telemetry. The main content area displays a table of distributions. One distribution is listed with the status 'Enabled', type 'Standard', domain 'd3kj2...', alternate domain 'zeeshancloudtech.online', origins 'zeeshancloudte', and last modified date 'September...'. Above the table are buttons for 'Enable', 'Disable', 'Delete', and 'Create distribution'. A search bar and filter type dropdown are also present.

Status	Descrip...	Type	Domai...	Alternate domain...	Origins	Last m...
Enabled	-	Standard	d3kj2...	zeeshancloudtech.online	zeeshancloudte	Septembe...

The screenshot shows the AWS WAF & Shield console with the 'Protection packs (web ACLs)' page selected. The left sidebar contains navigation links for WAF & Shield, AWS WAF, and AI traffic monetization. The main content area displays a table of protection packs. One pack is listed with the name 'zeeshan-cloudfront-waf', 19 rules, and buttons for 'Manage', 'View', and 'View'. Above the table are buttons for 'Actions' and 'Create protection pack (web ACL)'. A search bar and region scope dropdown are also present.

Name	Rules	Resources	Dashboard	AI traffic analysis	Sampled requ...
zeeshan-cloudfront-waf	19 rules	Manage	View	View	View

4. AWS Systems Manager Parameter Store

4.1 What is Parameter Store?

AWS Systems Manager Parameter Store provides secure, centralized storage for configuration values and parameters.

It can be used to store:

- Application configuration
- Environment variables
- Database configuration
- URLs
- Ports
- Feature flags
- Sensitive configuration using SecureString

Applications running on AWS can retrieve parameters using IAM permissions instead of hardcoding configuration values.

5. Parameter Store Implementation

5.1 Parameter Created

Created the following parameter:

Name:

/dev/app/db-passwd

Type:

SecureString

Tier:

Standard

Data type:

Text

Version:

1

Region:

us-east-1

Parameter ARN:

arn:aws:ssm:us-east-1:382736933724:parameter/dev/app/db-passwd

5.2 Parameter Hierarchy

The parameter uses a hierarchical naming structure:

/dev/app/db-passwd

The hierarchy can be used to organize parameters according to environment and application.

For example:

/dev/app/db-passwd

/dev/app/db-host

/dev/app/db-port

/qa/app/db-passwd

/qa/app/db-host

/qa/app/db-port

/prod/app/db-passwd

/prod/app/db-host

/prod/app/db-port

This makes parameter management easier across multiple environments.

6. SecureString and KMS

The parameter was created using:

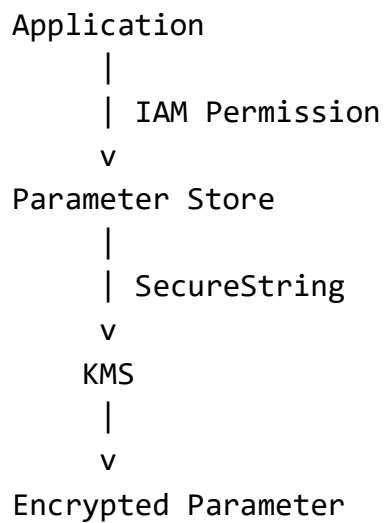
Type: SecureString

SecureString parameters are encrypted using AWS KMS.

For this lab, the AWS Systems Manager KMS key was used:

alias/aws/ssm

Conceptually:



Sensitive configuration should not be hardcoded inside application source code, Docker images, Jenkinsfiles, or configuration repositories.

7. Parameter Store Validation

Parameter Store parameters can be retrieved through AWS CLI.

Example:

```
aws ssm get-parameter \  
  --name "/dev/app/db-passwd" \  
  --region us-east-1
```

For a SecureString parameter, decryption can be requested using:

```
aws ssm get-parameter \  
  --name "/dev/app/db-passwd" \  
  --with-decryption \  
  --region us-east-1 \  
  --query "Parameter.Value" \  
  --output text
```

Parameters under the application path can also be checked using:

```
aws ssm get-parameters-by-path \  
  --path "/dev/app" \  
  --recursive \  
  --region us-east-1
```

us-east-1 +

```
~ $ aws ssm get-parameter \
> --name "/dev/app/environment"
{
  "Parameter": {
    "Name": "/dev/app/environment",
    "Type": "String",
    "Value": "dev",
    "Version": 1,
    "LastModifiedDate": "2026-09-30T06:53:12.258000+00:00",
    "ARN": "arn:aws:ssm:us-east-1:382736933724:parameter/dev/app/environment",
    "DataType": "text"
  }
}
~ $
```

CloudShell Actions

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

32°C Sunny

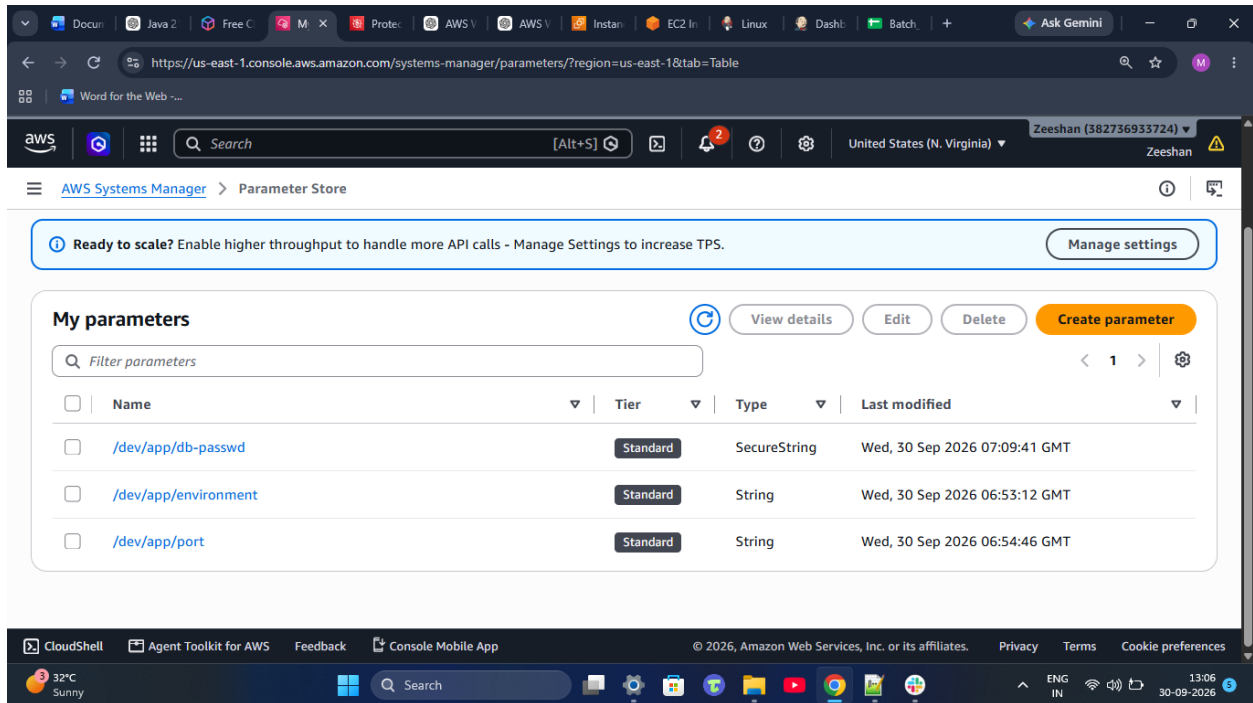
us-east-1 +

```
~ $ aws ssm get-parameter \
> --name "/dev/app/db-passwd" \
> --with-decryption \
> --region us-east-1 \
> --query "Parameter.Value" \
> --output text
DevPassword@123
~ $
```

CloudShell Actions

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

32°C Sunny



8. DevOps Use Cases

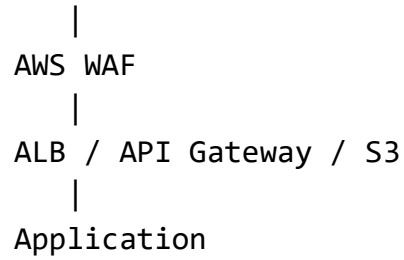
AWS WAF

AWS WAF can be used by a DevOps Engineer to:

- Protect CloudFront applications
- Block unwanted IP addresses
- Control request rates
- Protect against common web attacks
- Monitor suspicious traffic
- Apply managed security rules
- Protect APIs and web applications

Example:

Internet
|
CloudFront

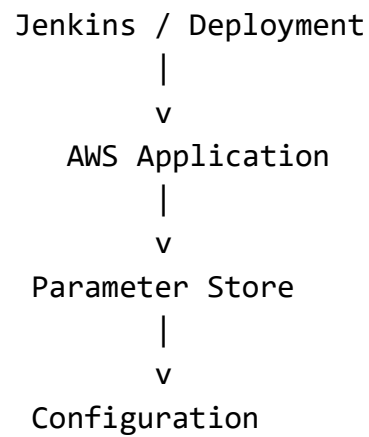


Parameter Store

Parameter Store can be used to:

- Centralize application configuration
- Separate configuration from application code
- Store environment-specific values
- Store encrypted configuration
- Provide configuration to EC2, Lambda, ECS, EKS and other workloads
- Control access using IAM

Example:



9. Key Observations

AWS WAF

- A WAF Web ACL can be associated with a CloudFront distribution.
- CloudFront WAF uses the **CloudFront/Global scope**.
- Managed rule groups provide predefined protections.
- Custom rules can be created for specific application requirements.
- Count mode is useful for observing traffic before enforcing blocking.
- Rate-based rules can help control excessive request traffic.

Parameter Store

- Parameters can be organized using hierarchical names.
- Standard and Advanced tiers provide different capabilities.
- SecureString provides encryption using AWS KMS.
- IAM should be used to control which applications or users can retrieve parameters.
- Configuration should be separated from application code.

10. Security Best Practices

- Use IAM roles instead of hardcoded AWS access keys.
- Follow least-privilege permissions for Parameter Store access.
- Use SecureString for sensitive configuration.
- Use managed WAF rules where appropriate.
- Test WAF rules in Count mode before enforcing them when possible.
- Monitor WAF metrics and sampled requests.
- Avoid storing passwords or secrets directly in source code.
- Review WAF rules regularly based on application traffic.

11. Conclusion

Successfully completed hands-on implementation and study of **AWS WAF** and **AWS Systems Manager Parameter Store**.

Implemented:

- AWS WAF protection pack
- CloudFront integration
- 19 WAF security rules
- Managed security protections
- Rate-based protection rules in Count mode
- AWS Systems Manager Parameter Store
- Hierarchical parameter naming
- SecureString parameter
- AWS KMS encryption integration
- AWS CLI commands for parameter retrieval and validation

The activity provided practical understanding of how WAF can protect a CloudFront-based application and how Parameter Store can securely centralize application configuration.